Relaciones entre Clases en Java

Taller Práctico Intensivo:

¡Bienvenidos al taller práctico de la Unidad 2.5! En este laboratorio intensivo aplicaremos las bases de la Programación Orientada a Objetos (POO) para modelar y desarrollar la **Arena de Héroes RPG**. El objetivo principal de esta jornada es dominar la traducción de diagramas de clases UML a código robusto en Java, comprendiendo cómo las relaciones de **Asociación**, **Agregación** y **Composición** dictan la estructura del software, el ciclo de vida de los objetos en memoria y la coherencia de datos de nuestras aplicaciones.

1. Arquitectura UML y Diagramación del Sistema RPG

1.1. Fundamentos de Diagramación UML y Multiplicidad

En la Unified Modeling Language (UML), modelar adecuadamente las interacciones entre clases es crítico para estructurar sistemas orientados a objetos coherentes. Las relaciones definen cómo interactúan los objetos, cómo se transfieren mensajes y de qué manera la vida de una entidad depende del contexto de otra.

La siguiente tabla sintetiza la notación UML utilizada para modelar el dominio de la Arena RPG:

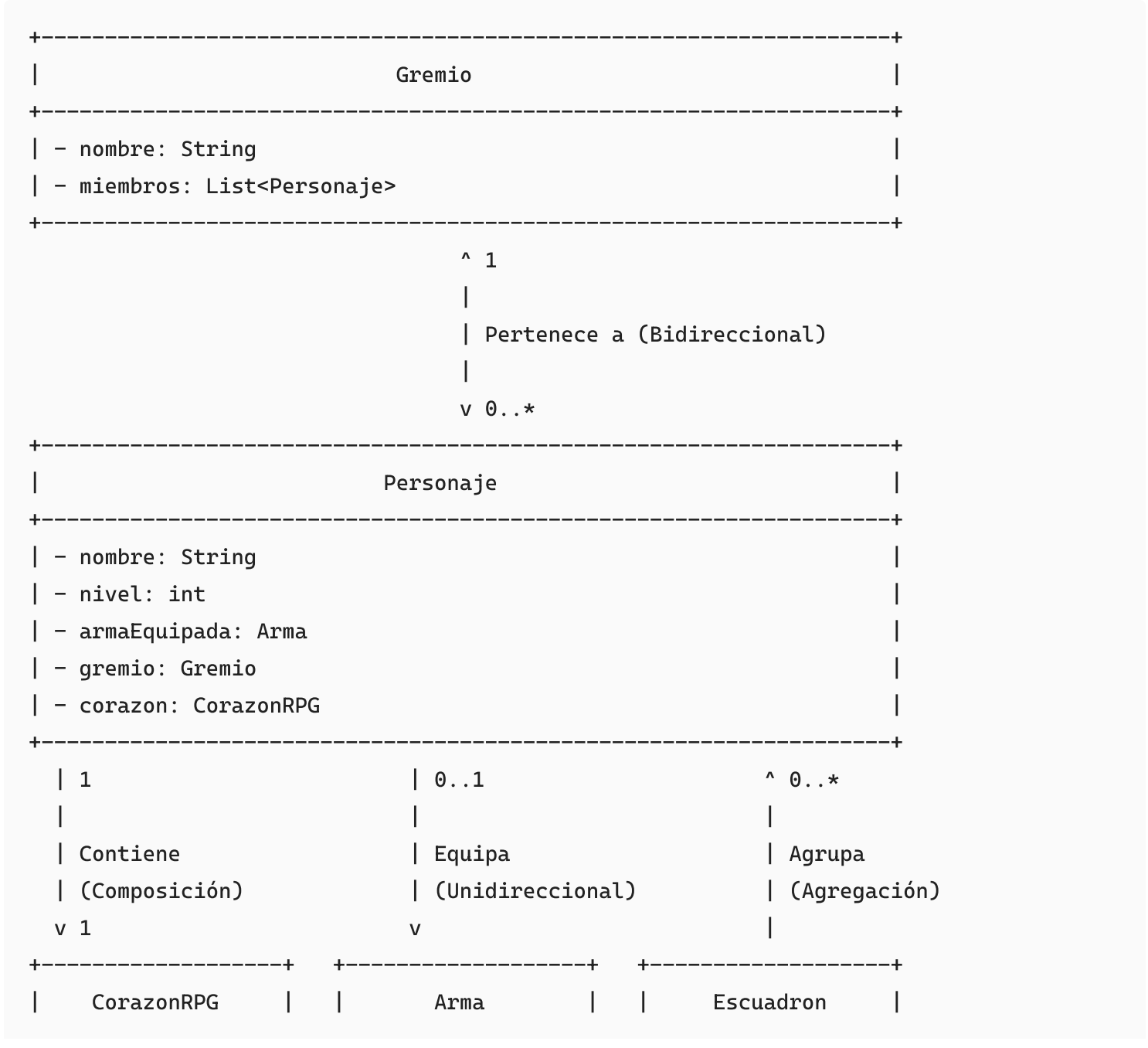
Concepto UML	Símbolo / Representación Textual	Cardinalidad / Multiplicidad	Significado en la Arena RPG
Asociación Unidireccional	`-------->`	`0..1`	Un `Personaje` puede tener opcionalmente equipada un `Arma` en un momento dado, pero el `Arma` no conoce a su dueño.
Asociación Bidireccional	`<------>`	`0..*` a `1`	Muchos `Personaje` pertenecen a un `Gremio`, y el `Gremio` mantiene la lista explícita de sus integrantes.
Agregación (Relación Débil)	`o------`	`0..*`	Un `Escuadron` agrupa múltiples `Personaje`. Si el escuadrón se disuelve, las instancias de los personajes continúan existiendo.
Composición (Relación Fuerte)	`*------`	`1..1`	Un `Personaje` posee un `CorazonRPG` de manera exclusiva. Si el personaje es destruido por la recolección de basura, su corazón se destruye con él.

Impacto Práctico: Agregación vs. Composición en Entidades de Software

- **Vinculación del Ciclo de Vida:** En la **Agregación**, los objetos constituyentes tienen un ciclo de vida independiente de la clase contenedora. En la **Composición**, el objeto hijo existe únicamente en función del objeto padre; no puede sobrevivir de forma autónoma.
- **Responsabilidad de Instanciación:** En una **Composición**, el objeto contenedor es responsable directo de crear las instancias de sus componentes dentro de su propio constructor. En una **Agregación**, los objetos contenidos son creados externamente y pasados como parámetros (vía constructor o métodos *setter*).
- **Control de Referencias y Encapsulamiento:** La **Composición** protege de manera estricta la referencia interna contra manipulaciones o suplantaciones desde el exterior, impidiendo que terceros reemplacen el componente o accedan a él sin la mediación de la entidad principal.

1.2. Especificación Visual del Sistema RPG

A continuación se presenta la especificación arquitectónica del dominio mediante un diagrama visual en formato ASCII, detallando la interacción entre los componentes del juego:



```
+------------------+   +------------------+   +------------------+
| - saludBase: int |   | - nombre: String |   | - nombre: String |
| - manaBase: int  |   | - danioBase: int |   | - miembros: List |
+------------------+   +------------------+   +------------------+
```

Requerimientos de Modelado de Entidades:

1. `**Personaje**` **equipa** `**Arma**` **(Unidireccional** `**0..1**`**):** La clase `Personaje` guarda un atributo de tipo `Arma`. Si el valor es `null`, el personaje combate desarmado.
2. `**Personaje**` **<->** `**Gremio**` **(Bidireccional** `**0..***` **a** `**1**`**):** Un personaje pertenece como máximo a un gremio, mientras que el gremio almacena múltiples referencias a sus personajes inscritos.
3. `**Escuadron**` **agrupa** `**Personaje**` **(Agregación** `**0..***`**):** El escuadrón mantiene una lista táctica de combatientes. La destrucción del escuadrón no altera el estado individual de sus integrantes.
4. `**Personaje**` **contiene** `**CorazonRPG**` **(Composición** `**1..1**`**):** Es el núcleo de atributos vitales del personaje. No existe personaje sin corazón, ni corazón sin personaje.

Transición Teórica a Código: Una vez definida la representación arquitectónica abstracta en el diagrama UML, el siguiente paso indispensable es plasmar estas reglas de estructura e interacción dentro de clases fuente en Java utilizando IntelliJ IDEA.

2. Implementación de Asociación Unidireccional y Bidireccional

2.1. Asociación Unidireccional: Equipamiento de Objetos (`Personaje` -> `Arma`)

En la asociación unidireccional, la clase origen mantiene un atributo que referencia a la clase destino, pero la clase destino desconoce la existencia del origen.

A continuación, complete la estructura básica de la clase `Personaje` para administrar el equipamiento de un `Arma`:

```java
public class Personaje {
    private String nombre;
    private int nivel;
    private Arma armaEquipada; // Asociación Unidireccional (0..1)

    public Personaje(String nombre, int nivel) {
        this.nombre = nombre;
        this.nivel = nivel;
        this.armaEquipada = null; // Inicialmente desarmado
    }
```

```
    public void equiparArma(Arma nuevaArma) {
        // TODO: Asignar 'nuevaArma' al atributo 'armaEquipada'.
        // TODO: Imprimir un mensaje confirmando el equipamiento del arma.
    }

    public void desequiparArma() {
        // TODO: Validar si 'armaEquipada' no es null.
        // TODO: Si tiene arma, reestablecer 'armaEquipada' a null e informar al
usuario.
        // TODO: Si no tiene arma, emitir un mensaje indicando que no hay arma para
retirar.
    }

    public int calcularAtaqueTotal() {
        int ataqueBase = this.nivel * 5;
        // TODO: Si 'armaEquipada' no es null, sumar su daño base al 'ataqueBase'.
        // TODO: Retornar el valor final calculado.
        return ataqueBase;
    }
}
```

¡ADVERTENCIA DE SEGURIDAD EN MEMORIA! Antes de invocar cualquier método o acceder a atributos de un objeto referenciado (como `armaEquipada.getDanioBase()`), es una obligación estricta verificar que la referencia no sea `null` mediante un bloque condicional (`if (this.armaEquipada != null)`). Omitir este control provocará de inmediato la excepción en tiempo de ejecución `NullPointerException`.

2.2. Asociación Bidireccional y Sincronización Cruzada (`Personaje` <-> `Gremio`)

Mantener la coherencia de las asociaciones bidireccionales en memoria requiere que ambos extremos de la relación se mantengan sincronizados explícitamente. Si un `Personaje` se une a un `Gremio`, el `Personaje` debe actualizar su atributo `gremio`, y simultáneamente el `Gremio` debe incluir al `Personaje` dentro de su lista de miembros.

Algoritmo de Sincronización en Doble Vía:

1. **Invocación Inicial:** El cliente llama a `personaje.unirseAGremio(nuevoGremio)`.
2. **Validación de Permanencia Previa:** `Personaje` verifica si ya está inscrito en un gremio diferente. Si es así, debe desvincularse invocando previamente el retiro de dicho gremio.
3. **Actualización de Referencia Propia:** `Personaje` asigna `nuevoGremio` a su atributo local `this.gremio`.

4. **Sincronización Cruzada Inversa:** `Personaje` invoca a `nuevoGremio.agregarMiembro(this)`, evitando la recursión infinita validando si el personaje ya se encuentra en la lista interna del gremio.

Complete las definiciones de sincronización bidireccional en el código de las clases correspondientes:

```
// Sincronización dentro de la clase Personaje
public void unirseAGremio(Gremio nuevoGremio) {
    // TODO: Validar si 'nuevoGremio' no es null y si es distinto del gremio actual.
    // TODO: Asignar el nuevo gremio al atributo local 'this.gremio'.
    // TODO: Invocación cruzada: Asegurar que 'nuevoGremio' registre este personaje en su lista.
}

// Sincronización dentro de la clase Gremio
public void agregarMiembro(Personaje p) {
    // TODO: Validar que 'p' no sea null.
    // TODO: Verificar si la lista de miembros no contiene ya a 'p'.
    // TODO: Añadir 'p' a la colección interna de miembros.
    // TODO: Sincronización cruzada: Si 'p.getGremio()' no es 'this', invocar p.unirseAGremio(this).
}
```

Reflexión sobre el Ciclo de Vida: Note cómo, a medida que avanzamos de referencias unidireccionales a asociaciones bidireccionales cruzadas, la vida de las entidades se entrelaza fuertemente en la memoria Heap de Java, demandando metodologías estrictas para el alta, baja y modificación de enlaces.

3. Agregación vs Composición en Java

3.1. Agregación: Gestión del `Escuadron` (Relación Débil)

La **Agregación** modela una relación del tipo "*tiene-un*" de carácter débil. Los componentes existen de forma ajena al contenedor. Si una instancia de `Escuadron` es eliminada o su referencia se reasigna a `null`, los objetos de la clase `Personaje` que la conformaban siguen plenamente activos e intactos en el sistema.

Desarrolle la clase `Escuadron` siguiendo los requisitos planteados:

```
import java.util.ArrayList;
import java.util.List;

public class Escuadron {
    private String nombre;
    private List<Personaje> miembros; // Agregación (0..*)
```

```
    public Escuadron(String nombre) {
        this.nombre = nombre;
        this.miembros = new ArrayList<>(); // Inicialización de la colección
    }

    public void reclutarPersonaje(Personaje p) {
        // TODO: Validar que 'p' no sea null y que no esté ya dentro de 'miembros'.
        // TODO: Agregar 'p' a la lista 'miembros' sin alterar la instanciación
original de 'p'.
    }

    public void expulsarPersonaje(Personaje p) {
        // TODO: Remover 'p' de la lista 'miembros' si existe.
        // TODO: Nota: 'p' mantiene su estado, vida y objetos intactos en memoria.
    }
}
```

3.2. Composición: Encapsulamiento del `CorazonRPG` (Relación Fuerte)

La **Composición** representa una relación de pertenencia absoluta y excluyente. El `CorazonRPG` es una parte constitutiva del `Personaje`. **Debe ser instanciado dentro del constructor** de `Personaje` y no debe existir ningún método público de tipo *setter* que permita reemplazar la instancia del corazón por una externa.

```
public class CorazonRPG {
    private int saludBase;
    private int manaBase;

    public CorazonRPG(int saludBase, int manaBase) {
        this.saludBase = saludBase;
        this.manaBase = manaBase;
    }

    // Getters y métodos de manipulación interna de vida/maná...
}
```

Tabla Comparativa de Implementación en Java

Criterio de Comparación	Agregación (`Escuadron` -> `Personaje`)	Composición (`Personaje` -> `CorazonRPG`)
Instanciación del Componente	Externa a la clase contenedora. Se recibe como parámetro en métodos o	Interna y explícita dentro del constructor de la clase contenedora.

	constructores.	
Dependencia del Ciclo de Vida	Independiente. La destrucción del `Escuadron` deja libres a los personajes.	Dependiente. La destrucción del `Personaje` implica la subsecuente pérdida del `CorazonRPG`.
Visibilidad de Setters	Permitidos. Se pueden agregar o reemplazar miembros libremente vía `setters` o métodos de lista.	**Prohibidos**. No debe existir un `setCorazonRPG()` que reemplace el objeto instanciado.

3.3. Multiplicidad y Colecciones en Java

Traducir la multiplicidad de un diagrama UML a código Java requiere seleccionar la estructura de almacenamiento adecuada según las características de la relación:

```
Multiplicidad UML (0..1 o 1..1)  --->  Atributo Directo (ej: private Arma arma;)
Multiplicidad UML (0..* o 1..*)  --->  Estructura de Colección (ej: List<Personaje>
miembros;)
```

- **Multiplicidad `**0..1**` o `**1..1**`:** Se declara un atributo del tipo directo de la clase asociada. Si es `0..1`, la referencia acepta valores `null`; si es `1..1`, debe garantizarse la inicialización inmediata.
- **Multiplicidad `**0..***` o `**1..***`:** Se utiliza una interfaz de colección (`List<T>`) instanciada con una clase concreta como `ArrayList<T>`. Esta estructura habilita la expansión dinámica en memoria.

Transición a la Arquitectura Global: Con las piezas teóricas e implementaciones individuales de cada tipo de relación listas, es el momento de unificar todas las clases dentro de un flujo ejecutable e interactivo de consola.

4. Reto Integrador: Evolución de la Arena RPG

4.1. Integración en el Menú Interactivo en Consola

El reto integrador requiere construir una interfaz textual de consola que permita al usuario gestionar la Arena RPG en tiempo real.

Requerimientos del Sistema:

1. **Control de Flujo:** Implemente un ciclo `do-while` respaldado por una estructura `switch` para navegar las opciones del sistema.
2. **Formateo de Salida:** Utilice `System.out.printf` para generar tablas y reportes formateados de las estadísticas de los personajes y sus escuadrones.
3. **Construcción Eficiente de Textos:** Emplee la clase `StringBuilder` dentro de los métodos `toString()` o métodos de reporte para concatenar eficientemente la información del estado del `CorazonRPG`, `Arma`, `Gremio` y `Escuadron`.

Plantilla del Menú Principal a Implementar:

```
import java.util.Scanner;

public class AplicacionArenaRPG {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int opcion = 0;

        // TODO: Instanciar colecciones globales para listar Gremios, Escuadrones y
Personajes libres.

        do {
            System.out.println("\n=== MENÚ ARENA DE HÉROES RPG ===");
            System.out.println("1. Crear Personaje");
            System.out.println("2. Crear Gremio");
            System.out.println("3. Reclutar Personaje a Escuadrón");
            System.out.println("4. Equipar Arma a Personaje");
            System.out.println("5. Mostrar Estado Detallado (Personaje y Corazón
RPG)");
            System.out.println("6. Salir");
            System.out.print("Seleccione una opción: ");
            opcion = scanner.nextInt();

            switch (opcion) {
                case 1:
                    // TODO: Solicitar nombre e instanciar un nuevo Personaje.
                    break;
                case 2:
                    // TODO: Solicitar nombre e instanciar un nuevo Gremio.
                    break;
                case 3:
                    // TODO: Seleccionar un Personaje existente y agregarlo a un
Escuadron (Agregación).
                    break;
                case 4:
                    // TODO: Instanciar un Arma y asignarla a un Personaje
(Asociación Unidireccional).
                    break;
                case 5:
                    // TODO: Construir reporte usando StringBuilder e imprimir
estadísticas con System.out.printf.
                    break;
```

```
                case 6:
                    System.out.println("Cerrando la Arena de Héroes...");
                    break;
                default:
                    System.out.println("Opción inválida.");
            }
        } while (opcion != 6);
    }
}
```

Transición a Depuración: Un código con complejas redes de objetos entrelazados puede ocultar desbordamientos de memoria o punteros nulos. A continuación aprenderemos a diagnosticar y solucionar estas fallas usando las herramientas de IntelliJ IDEA.

5. Puntos de Depuración y Diagnóstico en IntelliJ IDEA

5.1. Detección de Desbordamiento de Pila (`StackOverflowError`)

Un error clásico al implementar relaciones bidireccionales en Java ocurre al invocar recursivamente el método `toString()`. Si `Personaje.toString()` imprime los datos de su `Gremio`, y a su vez `Gremio.toString()` itera e imprime el `toString()` de cada uno de sus miembros, el programa entra en un ciclo infinito de llamadas cruzadas hasta agotar la pila de llamadas del sistema, lanzando un `java.lang.StackOverflowError`.

```
Personaje.toString() ---> Gremio.toString() ---> Personaje.toString() --->
StackOverflowError
```

Pasos para Diagnosticar y Solucionar con el Depurador de IntelliJ IDEA:

- **Fijar Breakpoints:** Coloque un punto de interrupción (Breakpoint) dentro de la primera línea del método `toString()` de la clase `Personaje`.
- **Ejecutar en Modo Debug:** Inicie la aplicación presionando el ícono de insecto o mediante la combinación `Shift + F9`.
- **Inspeccionar la Pila de Llamadas (Call Stack):** Abra la ventana inferior **Frames / Call Stack**. Si observa que las llamadas a los métodos `Personaje.toString()` y `Gremio.toString()` se repiten alternadamente cientos de veces, ha confirmado la recursión circular.
- **Solución de Código:** Rompa la cadena de llamadas en uno de los dos lados de la relación. En `Gremio.toString()`, limite la impresión de los miembros a únicamente sus nombres o identificadores simples, evitando volver a llamar al `toString()` completo del objeto `Personaje`.

5.2. Diagnóstico de Referencias Nulas y Desincronización

A continuación se detalla una guía de diagnóstico para los errores de memoria más frecuentes al administrar relaciones en Java:

Error / Excepción	Causa Raíz en Relaciones	Punto de Control / Herramienta en IntelliJ IDEA
`**NullPointerException**`	Invocación de un método sobre una referencia no instanciada (ej. `armaEquipada.getDanioBase()` cuando el arma es `null`, o falta de instanciación del `CorazonRPG` en el constructor).	Inspeccionar la ventana **Variables** en el panel de Debug. Evaluar si la referencia actual muestr el valor `null` antes de ejecución de la línea cor fallo.
Desincronización Bidireccional	Omitir la actualización cruzada (ej. actualizar `Personaje.gremio` sin agregar la instancia a `Gremio.miembros`).	Agregar un **Watch** (`Alt + F8`) sobre las expresiones `personaje.getGremio(` y `gremio.getMiembros()` para validar su consistencia mutua.
`**ConcurrentModificationException**`	Modificar una colección (`List<Personaje>`) con operaciones de adición o remoción mientras se la recorre mediante un ciclo `for-each`.	Usar un punto de interrupción condiciona dentro del ciclo y revisa el estado del cursor de l lista en la pestaña **Variables**. Reemplazar por un `Iterator` o iter sobre copias de la lista.

Conclusión y Siguiente Paso

Estimado estudiante, la correcta administración de referencias en Java es el pilar sobre el cual se construyen arquitecturas de software robustas y escalables. Le instamos a ejecutar su taller paso a paso, abriendo la pestaña **Variables** de IntelliJ IDEA y rastreando la memoria Heap en cada instanciación. ¡Nos vemos en la arena de combate!